



A procedure to detect problems of processes in software development projects using Bayesian networks



Mirko Perkusich^{a,*}, Gustavo Soares^a, Hyggo Almeida^a, Angelo Perkusich^b

^a Department of Computing and Systems, Federal University of Campina Grande, Rua Aprigio Veloso, 882, Bodocongo, 58109 900 Campina Grande, PB, Brazil

^b Department of Electrical Engineering, Federal University of Campina Grande, Rua Aprigio Veloso, 882, Bodocongo, 58109 900 Campina Grande, PB, Brazil

ARTICLE INFO

Article history:

Available online 20 August 2014

Keywords:

Software process simulation modeling
Bayesian networks
Software process management
Software development project

ABSTRACT

There are several software process models and methodologies such as waterfall, spiral and agile. Even so, the rate of successful software development projects is low. Since software is the major output of software processes, increasing software process management quality should increase the project's chances of success. Organizations have invested to adapt software processes to their environments and the characteristics of projects to improve the productivity and quality of the products. In this paper, we present a procedure to detect problems of processes in software development projects using *Bayesian networks*. The procedure was successfully applied to *Scrum*-based software development projects. The research results should encourage the usage of *Bayesian networks* to manage software processes and increase the rate of successful software development projects.

© 2014 Elsevier Ltd. All rights reserved.

1. Introduction

Over the years, software processes have evolved to take advantage of the organizations' structure and capabilities of human resources as well as the specific characteristics of the systems that they develop. There are several software processes models and methodologies such as waterfall, spiral and agile that are best suited depending on the project's and organizations' characteristics. For example, some would recommend waterfall for organizations with defined processes and large software development projects for a system with high criticality such as an online banking system. On the other hand, others would recommend agile for a project with a complex user interface and low criticality such as a social network application.

Even though software processes have evolved, there is still a low rate of successful software projects. According to a study performed in 2008 (Emam & Koru, 2008), only between 46 and 55 percent of IT projects succeed. For an applied discipline, this is a low success rate. Since software is the major output of software processes and software development projects, they are both correlated. Increasing the software processes management quality should increase the software development project's chances of success.

One approach to assess and manage software processes is through software process simulation modeling (Kellner, Madachy, & Raffo, 1999). In these approach, researchers construct models that represent software processes to assist management and increase the project's chances of success. With this purpose, model developers use techniques such as *system dynamics*, discrete events simulation and *Bayesian networks* (Zhang, Kitchenham, & Pfahl, 2008). Even though this approach has the potential to encompass key factors of software process models such as specification, quality and development (Sommerville, 2010), in most cases, researchers have used it for a limited scope of processes. For instance, Abouelela and Benedicenti (2010) and Jeet, Bhatia, and Minhas (2011a) applied this approach only for quality management. Furthermore, most studies do not present a procedure to use the model. A procedure to construct and use models to assist on software process management is essential to give model developers a common knowledge and instructions to optimize the chances of constructing a model that best suits the project that it will be applied to.

By modelling software development processes key factors, it is possible to assist on continuous improvement by detecting their problems. In this paper, as shown in Section 3, we present a procedure to construct and use *Bayesian networks* for this purpose. The proposed procedure can be applied to detect problems in any software development methodology or framework.

We used *Bayesian networks* due to their capability to handle uncertainty and also because of their ease of understanding and

* Corresponding author.

E-mail addresses: mirko.perkusich@copin.ufcg.edu.br (M. Perkusich), gsoares@computacao.ufcg.edu.br (G. Soares), hyggo@dsc.ufcg.edu.br (H. Almeida), perkusic@dee.ufcg.edu.br (A. Perkusich).

modification by practitioners. Researchers have applied this technique for expert systems in areas such as software maintenance project management (de Melo & Sanchez, 2008), safety control in complex project environments (Zhang, Wu, Ding, Skibniewski, & Yan, 2013) and performance forecast of innovation projects (de Oliveira, Possamai, Dalla Valentina, & Flesch, 2012).

To validate the procedure, we applied it to *Scrum*-based software development projects. We chose *Scrum* because it is the most popular agile framework (VersionOne, 2013), has an active community and specialized organizations that support its state of art such as *Scrum Alliance* and *Scrum.org*. Our validation reasoning is similar to Lee, Park, and Shin (2009), in which the authors present a procedure to construct and use *Bayesian network* for risk management in large engineering projects and validate it by applying it to a specific purpose: the Korean shipbuilding industry. We divided the procedure validation into two steps: (i) validate the *Bayesian network* and (ii) its usage process. To validate the *Bayesian network*, as shown in Section 4.6.1, we individually tested its node probability tables and used simulated scenarios to test its outputs. To validate the *Bayesian network* usage process, as shown in Section 4.6.2, we performed a case study for two real projects in a company located in Brazil. For both cases, it helped to improve the quality of the processes thereby improving the project's chances of success with positive cost-benefit.

This paper is organized as follows: in Section 2, we present relevant literature review focusing on software processes evolution, software process simulation modelling, *Bayesian networks* and *Scrum*; in Section 3, we present the procedure in details and a guideline to build *Bayesian networks* for software processes; in Section 4, we present an overview of the procedure's application to *Scrum*-based software projects, which Perkusich, de Almeida, and Perkusich (2013a) presents in detail, and information regarding its validation which encompassed the *Bayesian network* and its usage process; and, in Section 5, we present our conclusions, current limitations and future works.

2. Literature review

2.1. Software processes

According to Sommerville (2010), software processes are interleaved sequences of technical, collaborative, and managerial activities with the overall goal of specifying, designing, implementing, and testing a software system. Furthermore, they can be described with roles, products, and pre- and post-conditions. According to Boehm (1988), they provide guidance on the order in which a project should carry out its major tasks. Over the years, software processes have evolved to handle the expectations of the software development projects in the best way possible. As a consequence, several software process models and methodologies were proposed in an attempt to increase the chances of success of the software development projects. Each of these had their advantages and disadvantages.

According to Boehm (2006), during the early years of software engineering in the 1950s, software processes were plan-driven and sequential because software projects were managed as hardware projects. During the 1960s, because software could be easily modified, many programmers started to use the "code and fix" approach and this created heavily patched spaghetti code.

In the 1970s, as a reaction to the problems caused by the "code and fix" approach, waterfall and formal methods were proposed. The waterfall process represents the fundamental process activities as process phases (Royce et al., 1970). It was intended to be iterative but it was interpreted as sequential. Formal methods are mathematical techniques, often supported by tools, for

developing software and hardware systems. Mathematical rigor enables users to analyze and verify these models at any part of the program life-cycle. These parts are: requirements engineering, specification, architecture, design, implementation, testing, maintenance, and evolution (Woodcock, Larsen, Bicarregui, & Fitzgerald, 2009). As a reaction to these heavy-weight processes, the Rapid Application Development (RAD), which is incremental and iterative (Martin, 1991), was proposed.

Later in the 1980s, software process standards were proposed to avoid process noncompliance and Software Capability Maturity Models were used to assess an organization's software process maturity. The Software Capability Maturity Model (SW-CMM) and ISO-9001 were created and largely used. The SW-CMM provides an effective framework for both capability assessment and improvement (Humphrey, 1989). ISO-9001 is part of the ISO-9000 family of standards that is related to quality management systems. It is not specific for software development, but it was largely used for external quality assurance of software development projects. In 1987, Osterweil (1987) proposed the usage of programming techniques and formalisms to express software process descriptions. Furthermore, during the 1980s new software development processes were proposed such as evolutionary (McCracken & Jackson, 1982), *Cleanroom* (Mills, Dyer, & Linger, 1987) and risk-driven spiral (Boehm, 1988).

During the 1990s, due to the need to reduce time-to-market, a major shift occurred away from sequential models towards agile methods such as *Adaptive Software Development* (ASD) (Highsmith, 1999), *Crystal* (Cockburn, 2001), *Dynamic Systems Development* (DSDM) (Stapleton & Constable, 1997), *eXtreme Programming* (XP) (Beck, 2000), *Feature Driven Development* (FDD) (Coad, de Luca, & Lefebvre, 1999), *Kanban* (Anderson, 2010), *Scrum* (Cohn, 2009) and *Scrumban* (Ladas, 2009). This approach relies on lightweight processes with an incremental approach to software specification, development, and delivery to maximize value delivery to the customers. It intends to deliver working software quickly to users, who can then propose new and changed requirements to be included in later iterations of the system (Sommerville, 2010). On the other hand, even though lightweight (agile) software processes arose, heavyweight (plan-driven) software processes were still used and new ones were proposed in the late 1990s and the following years such as the *Unified Software Development Process* (Jacobson, Booch, & Rumbaugh, 1999) and the *Rational Unified Process* (RUP) (Kruchten, 2003). In general, the choice of the best suited software process for a project depends on the type of product, size of the project, and the business requirements. According to Sommerville (2010), agile methods are best suited for small or medium-sized projects with low criticality. Plan-driven methods are best suited for large companies with defined processes and large projects.

Software companies have paid greater attention as to improve process productivity and quality of the delivered products. As a consequence, the evaluation of software processes became a very important issue because software is its major outcome (Li, Li, Wu, & Song, 2012). Software processes can be improved by process standardization. This leads to improved communication, a reduction in training time, and also makes automated process support more economical.

2.2. Software process simulation modeling

Another way to improve software processes is through software process simulation modeling because it can support software process management (Kellner et al., 1999). The goal of software process simulation modeling is to use technologies, people, and tools to collaboratively increase the chances of success of software development projects. These models may focus on development

processes, software maintenance, and evolution. They can represent processes as they are now or as planned for a future implementation. They are abstractions that only represent factors judged as important by the model's developer. These factors may be defined empirically or through the opinions of experts (Melis, 2006). According to Zhang et al. (2008), the most popular techniques to model software processes are *system dynamics* and *discrete events simulation*. Even though this approach has the potential to encompass key factors of software process models such as specification, quality and development (Sommerville, 2010), in most cases, researchers have used it for a limited scope of processes. For example, researchers applied software process simulation modeling to several areas such as software trustworthiness (Bai, Huang, Zhang, & Koolmanojwong, 2012), risk management (Büyükoçkan & Ruan, 2010; Fan & Yu, 2004; Hearty, Fenton, Marquez, & Neil, 2009; Houston, Mackulak, & Collofello, 2001; Jeet, Bhatia, & Minhas, 2011b), and quality management (Aboueila & Benedicenti, 2010; Jeet et al., 2011a). Furthermore, most studies do not present a procedure to use the model. A procedure to construct and use models to assist on software process management is essential to give model developers a common knowledge and instructions to optimize the chances of constructing a model that best suits the project that it will be applied to.

Bai et al. (2012) used a hybrid technique to build a stakeholder-oriented, software development process model to increase the trustworthiness in the project's delivery. The researchers identified the stakeholders and their perspectives to model the software development process. Some of the identified stakeholders were: the client, the process engineer, the specialist, and the process manager. Some of the identified perspectives were: the workforce modeling, team composition, the project's business case analysis, and the system's architecture. The researchers used discrete and continuous techniques to build the model so it could adapt to the internal and external changes of the process. The model was successfully applied to ISPW-6 (Kellner et al., 1991) using Little-JIL (Osterweil, 1998). Compared to our work, Bai et al. (2012) used a different technique. Furthermore, they do not provide instructions regarding the usage of the model in a real project. The procedure we propose uses *Bayesian networks* and provides instructions regarding the model's usage, as shown in Section 3.

Fan and Yu (2004), Hearty et al. (2009), Houston et al. (2001), Büyükoçkan and Ruan (2010), Fenton, Krause, and Neil (2002) and Jeet et al. (2011b) modeled software processes to support risk management. Fan and Yu (2004) used *Bayesian networks* to build a model capable of predicting potential risks, identifying the source of risks, and supporting dynamic resource adjustment. Hearty et al. (2009) used *Bayesian networks* to model the XP process and evaluate risks as well as to quantitatively estimate the effort necessary to finalize the project. Houston et al. (2001) used *system dynamics* and the *stochastic simulation* to model risk factors and simulate their effects to support risk management activities. Büyükoçkan and Ruan (2010) used a special *fuzzy operator* – Choquet integral – to model various effects of importance and interactions among risks. Fenton et al. (2002) showed how to use a *Bayesian network* to predict software defects and perform “what if” scenarios. Jeet et al. (2011b) used *Bayesian networks* to estimate the impact of low productivity on the schedule of a software development. They used interviews and historical data to build the model. These works present models that have different purposes than the procedure we propose. Our procedure focus on supporting continuous improvement, as shown in Section 3.

Aboueila and Benedicenti (2010), Jeet et al. (2011a), Engel and Last (2007) and Yuen and Lau (2011) modeled software processes to support quality management. Aboueila and Benedicenti (2010) used *Bayesian networks* to predict a releases rate of defects in a XP project. Furthermore, the model predicts the project's duration. In

this way, the proposed model allows its users to determine if a XP project will succeed. The inputs of the model are: information on requirements, size of the team, *pair programming* usage, *Test-Driven Development* usage, client availability, team velocity, and the number of defects. Jeet et al. (2011a) used *Bayesian networks* to detect the number of defects in a software development project. The rate of defects and the project manager's judgments are used for predictions and support in managing the number of defects. Engel and Last (2007) presented a model to estimate software testing costs and risks using the fuzzy logic paradigm. Yuen and Lau (2011) proposed a fuzzy group analytical hierarchy process approach using ISO/IEC9126-1:2001 criteria to evaluate the quality of in-house or third party developed software. As already mentioned, these works have limited scope and focus on making predictions to support quality management. Our work encompasses the key software processes factors to assist on continuous improvement.

Pendharkar, Subramanian, and Rodger (2005), Kouskouras and Georgiou (2007), Ahmed and Muzaffar (2009) and de Melo and Sanchez (2008) modeled software processes to support project planning. Pendharkar et al. (2005) used *Bayesian networks* to predict the effort necessary to complete a software development project using historical data and subjective estimations of experts. Kouskouras and Georgiou (2007) built a discrete simulation event model to estimate several software project details such as delivery dates and quality metrics. Ahmed and Muzaffar (2009) presented an effort prediction framework that is based on type-2 fuzzy logic to allow handling imprecision and uncertainty inherent in the information available for effort prediction. de Melo and Sanchez (2008) presented a *Bayesian network* for maintenance project delays based on the experience of specialists and a corresponding tool to help in managing software maintenance projects. On the other hand, our procedure should be used during the execution of software development projects to detect problems of software processes.

Settas, Bibi, Sfetsos, Stamelos, and Gerogiannis (2006), Stamelos (2010), Stamelos, Angelis, Dimou, and Sakellaris (2003) and Spasic and Onggo (2012) modeled software processes to support other project management activities. Settas et al. (2006) and Stamelos (2010) used *Bayesian networks* to help managerial decision making by modeling software project management anti-patterns. Stamelos et al. (2003) modeled the uncertainties of factors to estimate software productivity. Spasic and Onggo (2012) used *agent-based simulation* to model the software process of companies with CMMI level 3 and that also use the Rational Unified Process. The model's goal is to estimate the project duration given the system's components and human resources. To validate the model, Spasic and Onggo (2012) compared the model's calculated data with the real data of past projects and obtained success.

Kahen, Lehman, Ramil, and Wernick (2001) described a *system dynamics* approach to investigate the consequences on long-term evolution of software processes due to decisions made by the managers of these processes. This work aims to contribute to the development of techniques for strategic management, planning, and the control of long-term software-product evolution. Our work has a different purpose and used *Bayesian networks*. The advantages of using *Bayesian networks* is shown in Section 2.3.

Uzzafer (2013) proposed a system dynamics and discrete event simulation-based model for the strategic management of software projects. It estimates the best strategic management, considering budget and schedule, given the project's risk and cost commitments. The approach allows users to handle custom risk management models, software cost models, and also project management tools. On the other hand, the model, differently from the procedure we propose, does not consider quality and scope requirements.

Melis (2006), Wu and Yan (2009) and Kuppaswami, Vivekanandan, Ramaswamy, and Rodrigues (2003) modeled software processes to evaluate the impact of some XP practices in software projects. Melis (2006) used *discrete events simulation* and *system dynamics* to model the XP process in order to evaluate the impact of *pair programming* and *Test Driven Development* on the evolution of XP-based projects. Wu and Yan (2009) used *system dynamics* to model the XP process as to evaluate the impact of *pair programming* on the results of the project. This was used to demonstrate its importance to undergraduate students. Kuppaswami et al. (2003) used *system dynamics* to model the XP process, with all XP practices described by Beck (2000), to evaluate the project's effort. The goal was to verify if XP usage reduces the cost to develop a project compared with traditional methodologies. None of these works used *Bayesian network* to model software processes. Our procedure can be applied to any software process model, as shown in Section 3.

Nagy, Njima, and Mkrtchyan (2010) used *Bayesian networks* to build a model to evaluate the key factors of an agile project in order to calculate its health. The model supports decision making and the detection of problems. The proposed model used the knowledge of experts, but did not inform how the knowledge was used to build the *Bayesian network*. Furthermore, the model was not validated. Since this model assesses the project's health, we consider that it has a goal similar to ours. On the other hand, given its weaknesses and the lack of instructions on how to use it, we believe that it has limited contribution. Even though our procedure was validated with *Scrum*, as shown in Section 4, it can be applied to methodology or framework.

Many works used *Bayesian networks* to model software processes and, as shown in Section 4.6.1, several works proposed methods to construct *Bayesian networks*. We provide a guideline, as shown in 3, to construct a *Bayesian network* to detect problems and processes in software development projects, but, if preferable, a different method can be used. Anyway, the guideline should serve to give practitioners a common knowledge regarding constructing *Bayesian networks* with this work's purpose. It is important to notice that, having the *Bayesian network* is not enough. As it is with any tool, understanding how to use it is necessary. So, we propose a procedure containing the necessary steps to, in practice, use the *Bayesian network* to assist on continuous improvement.

Our work presents a procedure to construct and use *Bayesian networks* to detect problems of processes in software development projects. The constructed *Bayesian network* should be used to support decision making regarding software process practices during the project's execution as to support continuous improvement. None of the cited work, except for Nagy et al. (2010), has a similar goal for using *Bayesian networks*. On the other hand, their work has weaknesses that limits its contribution. The innovative contributions of our work are:

- A guideline to construct *Bayesian networks* to model software development processes key factors to assist on continuous improvement by detecting their problems.
- A procedure to use the *Bayesian network* on software development projects. The procedure can be applied to detect problems in any software development methodology or framework.
- Novel results regarding the application of the proposed procedure on *Scrum*-based software development projects.

2.3. Bayesian networks

Bayesian networks belong to the family of probabilistic graph models and are used to represent knowledge about an uncertain domain (Ben-Gal, 2007). A *Bayesian network*, B , is a directed acyclic

graph that represents a joint probability distribution over a set of random variables V (Friedman, Geiger, & Goldszmidt, 1997). The network is defined by the pair $B = \{G, \Theta\}$. G is the directed acyclic graph in which the nodes $X_1, \dots, X_n$ represent random variables and the arcs represent the direct dependencies between these variables. Θ represents the set of the probability functions. This set contains the parameter $\theta_{x_i|\pi_i} = P_B(x_i|\pi_i)$ for each x_i in X_i conditioned by π_i , the set of the parameters of X_i in G . Eq. (1) presents the joint distribution defined by B over V .

$$P_B(X_1, \dots, X_n) = \prod_{i=1}^n P_B(x_i|\pi_i) = \prod_{i=1}^n \theta_{x_i|\pi_i} \quad (1)$$

We present an example of a *Bayesian network* in Fig. 1. Circles represent the nodes and arrows represent the arcs. The probability functions are usually represented by tables. Even though the arcs represent the casual connection's direction between the variables, information can propagate in any direction (Pearl, 1995).

Bayesian networks have many advantages such as suitability for small and incomplete data sets, structural learning possibility, combination of different sources of knowledge, explicit treatment of uncertainty, support for decision analysis, and fast responses (Uusitalo, 2007). Therefore, they are applied to support systems with uncertainty (Lee et al., 2009). *Bayesian networks* have been used for several expert systems such as assisting in safety decision making in complex project environments (Zhang et al., 2013) and to predict performance in innovation projects given their transformational leadership characteristics (de Oliveira et al., 2012). According to Ziv, Richardson, and Klosch (1996), the graphical structure of *Bayesian networks* is consistent with the uncertainty of software systems and therefore, it can be used to model software projects. Furthermore, the graphical interface of *Bayesian networks* facilitates communication with practitioners and makes it easy to be analyzed or modified by them.

On the other hand, for large *Bayesian networks*, building the directed acyclic graph and defining the probability functions are two significant barriers (Neil, Fenton, & Nielson, 2000). Since this paper's goal is not about constructing *Bayesian networks*, we will give a brief introduction to this topic. We will point to some references and propose a guideline to build *Bayesian networks* for software processes. Building the *Bayesian network's* directed acyclic graph is a risky process and requires a systems engineering process to manage it (Mahoney & Laskey, 1996). The choice of process centers around management of risk. Risky projects should use an iterative process and more established problems should use a sequential life-cycle process (Neil et al., 2000). Techniques, such as module and object oriented (Koller & Pfeffer, 1997) and fragments (Laskey & Mahoney, 1997), are used to define the *Bayesian network's* directed acyclic graph. Frameworks for building large *Bayesian networks* are described in the work of Neil et al. (2000) and Bangs and Wuillemin (2000).

The *Bayesian network's* probability functions are usually represented as node probability tables. The two forms to collect data and define node probability tables are through databases and through domain experts (Perkusich, Perkusich, & Almeida, 2013b). Defining node probability tables from databases can be automated by a process called *batch learning* (Heckerman, 1999). However for many practical problems one rarely finds an adequate database. Manually defining node probability tables through domain experts can become unfeasible depending on the number of nodes and states. As shown in the work of Fenton, Neil, and Caballero (2007), all kinds of inconsistencies could occur if domain experts try to elicit exhaustively the node probability table for a node with a large number (e.g., 125) of states. There are several methods to reduce this complexity and to encode expertise in large node probability tables. Noisy-OR (Huang & Henrion, 1996) and

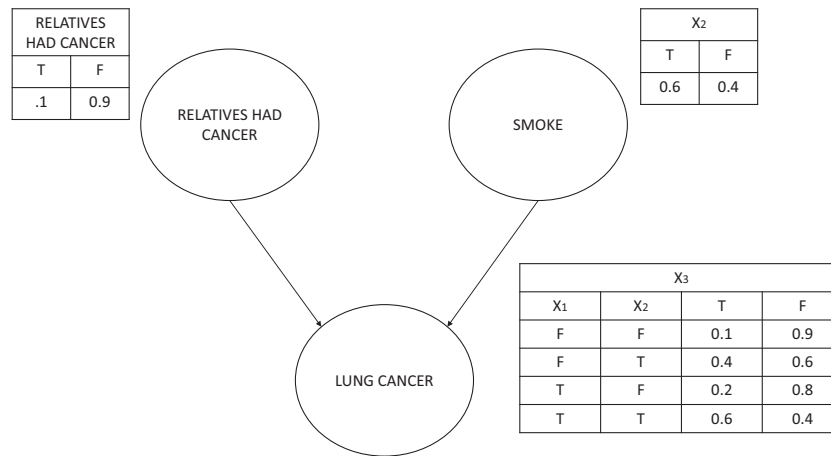


Fig. 1. A Bayesian network example.

Noise-MAX (Diez, 1993) are well established methods but Noisy-OR only applies to Boolean nodes, and Noisy-MAX does not model the range of relationships we seek here. Das (2004) proposed an algorithm to populate node probability tables while easing the extent of knowledge acquisition. Fenton et al. (2007) proposes an approach for Bayesian networks composed of ranked nodes. The approach is based on the doubly truncated Normal distribution with a central tendency that is invariably a type of weighted function of the parent nodes. Perkusich et al. (2013b) extends the work of Fenton et al. (2007) by proposing a method to collect data via a survey and statistically analyze the collected data to build the node probability tables.

2.4. Scrum overview

Scrum is the most used agile framework in the industry (VersionOne, 2013) and it is common to combine it with other agile frameworks such as eXtreme Programming (Cohn, 2009) and Kanban (Anderson, 2010) (a process called ScrumBan). Scrum is an iterative and incremental process to optimize the ability to foresee and control risks. The Scrum process is sustained by three pillars: transparency, inspection and adaptation (Griffiths, 2012). At the end of each iteration, called *sprint*, a functional product increment is delivered to be verified and validated by the stakeholders. Important aspects of the process such as acceptance criteria must be visible (i.e., transparent) to all stakeholders. The stakeholders should inspect the artifacts and progress of the project frequently to detect any undesired variability. Finally, the project should be adapted to support changes. We present an overview of Scrum in Fig. 2.

There are two main artifacts: product backlog and sprint backlog. The product backlog is an ordered list that represents the product's functionalities, requirements, improvements, and bug fixes. It should be ordered given value, risk, business priority, and necessity. Furthermore, it is emergent and should adapt to business changes (Pichler, 2010). The sprint backlog is composed of product backlog items that were allocated to a given sprint (Griffiths, 2012).

There are three roles: Product Owner, ScrumMaster, and developer (Griffiths, 2012). The Product Owner is responsible for maximizing the product's value. He/she should serve as an interface between the technical team and the business team and is the only one responsible for managing the product backlog. The ScrumMaster serves as a servant-leader and is responsible to ensure that Scrum's theories, rules, and practices are correctly applied in the project. The developers are responsible for executing any activity related to delivering the product increment at the end of the sprints (e.g., design, implement and test).

There are four essential meetings: planning, daily, review, and retrospective (Griffiths, 2012). The planning meetings occur at the beginning of each sprint to define its work and goal. The daily meetings occur daily to inspect the progress of the sprint and synchronize the team's work to mitigate risks. The review meetings occur at the end of each sprint to inspect the produced product increment and if necessary, to adapt the product backlog to changes requested by the business team. The retrospective meetings occur after the review meetings to assess the interactions between people, relationships, processes and tools. They identify problems, define action points (i.e., corrective and preventive actions), and define a plan to apply the action points.

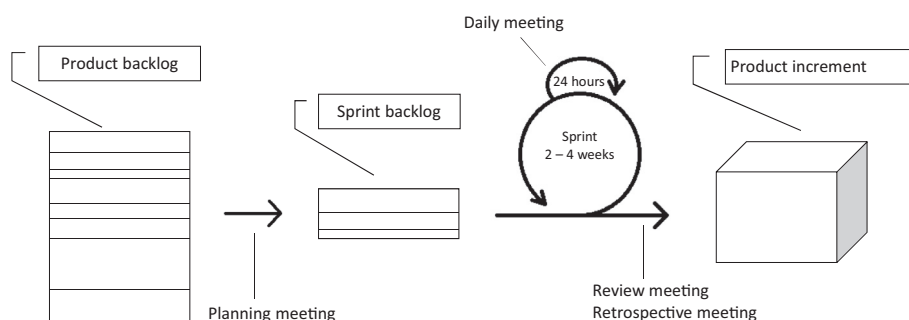


Fig. 2. Overview of Scrum.

3. Procedure to model software development projects to detect problems of processes using a Bayesian network

As mentioned earlier, a *Bayesian network* can be used to model software processes to aid practitioners and increase the chances of success of a software development project. With a *Bayesian network* it is possible to construct a cause-consequence relation visually and provide conditional probabilistic estimations of the software process's factors status. In this study, we propose a procedure to detect problems in a software development project's software processes usage by applying a *Bayesian network*, presented in Fig. 3.

3.1. Stage 1: Bayesian Network Construction

At this stage, the *Bayesian network* is constructed. The problem of constructing the *Bayesian network* can be divided into two parts: building the directed acyclic graph and defining the probability functions. The directed acyclic graph represents the project's software processes key factors and their relationships. A software process key factor is a factor related to the software process that the model developer judged as relevant for the model such as *Software validation* and *Software specification*. The probability functions represent the relative intensity between the relationships that have the endpoint in common. If the model developer uses an existing *Bayesian network* that models a software process as a base for the software process he/she needs to model, he/she may skip this stage. An example is if a project uses *eXtreme Programming* and a *Bayesian network*. Since *eXtreme Programming* already exists, it can be used as a base for the project's *Bayesian network* and this stage may be skipped. This is one of the most relevant contributions of this approach. Given that projects usually follow one or a combination of conventional software development methodologies or frameworks (e.g., *eXtreme Programming*, *RAD* or *Scrum*), if a *Bayesian network* exists for the given methodology or framework it should be easy to modify the *Bayesian network* to suit a project that uses this methodology or framework, or a similar one. In this case, the modifications would occur during the next stage (i.e. Stage 2: *Bayesian Network Evaluation*). The model developer should decide if the cost-benefit of modifying an existing *Bayesian network*, and skipping this step, is higher than creating one from scratch.

On the directed acyclic graph, each node represents a key software process factor and there is an edge between two nodes whenever they relate to each other. For each edge, the influenced key software process factor on the relationship is the edge's endpoint. For instance, we could say that the nodes *Software validation* and *Software specification* point to the node *Software process quality*. Furthermore, each key software process factor has possible states and each one of them has an associated probability. We could say that the node *Software process quality* has three possible states: *Low*, *Medium* and *High*, and that each has a probability of 33%. Given that, each key software process factor represents a set of tuples $N = \{(s_1, p_1), \dots, (s_{|N|}, p_{|N|})\}$, where s_i is a possible state and p_i is the associated probability. We also have the set of key software process factors $F = \{N_1, \dots, N_{|F|}\}$. Furthermore, we have the set of edges $R = \{(N_j, N_k) | N_j \subset F \wedge N_k \subset F\}$, where N_j is the initial point and N_k is the endpoint of the edge. The problem then is to find all elements of the sets F and R . To find all elements of the set F , we need to identify all key software process factors N_a . Then for each key software process factor N_a , find all the possible states s_i , and associated probability, p_i , where $a \leq |F|$ and $i \leq |N_a|$. Finally, to find all elements of the set R , we need to identify all f_j and f_k where f_j and $f_k \in F$.

A probability function can be represented by $Pr = \{A|B\}$ where A is a dependent variable and B is a set of parent nodes. Thus, the set $P = \{Pr_1, \dots, Pr_{|P|}\}$ represents all the probability functions for the *Bayesian network*. The problem is to find all the elements of the set P . To find them, for each Pr_i where $1 \leq i \leq |P|$, we need to quantify the relationship between A and B_j , where $1 \leq j \leq |B|$ and $B_j \in B$. As stated in Section 2.3, probability functions can be represented by weighted functions. For instance, given that *Software validation* and *Software specification* point to the node *Software process quality*, we need to define a weighted function for the *Software process quality*. As an example, we could say that $SPQ = 2 * SV + SS$, in which SPQ is *Software process quality*, SV is *Software validation*, and SS is *Software specification*.

Independent of the approach used to construct the *Bayesian network*, our goal is to have a *Bayesian network* that represents the software processes intended to be managed. As stated earlier, there are several software processes models such as waterfall, spiral, and *eXtreme Programming*. In our approach, each software process model should have a particular *Bayesian network*. Furthermore, companies usually modify existing software processes models to

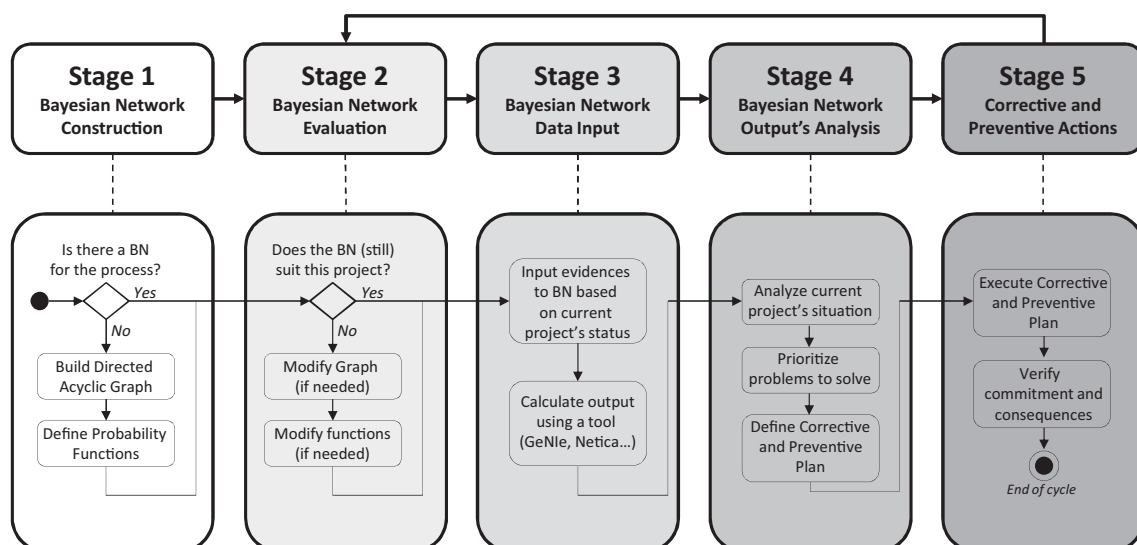


Fig. 3. Procedure to detect problems in a software development project's software processes usage using a Bayesian network.

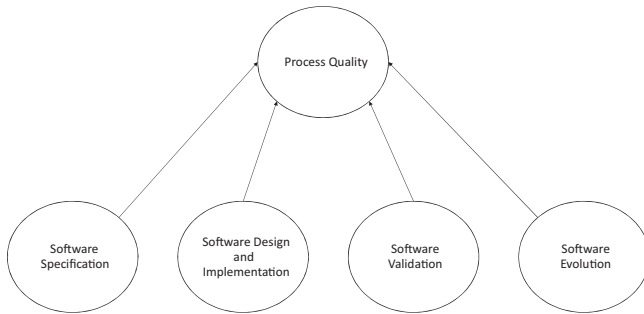


Fig. 4. Directed acyclic graph considering Sommerville's fundamental software processes activities.

suit a particular projects needs. Theoretically, each project should have a *Bayesian network*. Of course similar projects should have similar *Bayesian networks*. Given that a project uses a particular software process model, the effort to modify an existing *Bayesian network* for the given software process model to suit the given project should be low.

Even though the model developer can use any approach to construct the *Bayesian network*, we propose a guideline. The first step towards constructing a *Bayesian network* is to build the directed acyclic graph. For software processes simulation modeling, it is to identify the key factors and their relationships for the given software processes. It is necessary to identify the software processes' main activities and decompose them into activities that can be measured. In some cases, roles and products should also be identified. The first step to building the directed acyclic graph is to define the output node. Since we intend to assess the software process quality, we recommend to set it as the output node. According to Sommerville (2010), even though there are many different software processes, all must include four activities that are fundamental to software engineering: software specification, software design and implementation, software validation, and software evolution. This could be used to set *Software process quality's* parents. We present the resulting directed acyclic graph for the cited nodes in Fig. 4. Afterwards, it is necessary to decompose the parent nodes. In other words, it is necessary to identify activities, roles, or products that influence the identified key software process factors (e.g., *Teamwork skills* and *Software engineering technical practices* could be *Software design and implementation's* parent nodes). The same process continues until all parent nodes can be measured.

The next step is to define the node probability tables. For this step, we use the approach proposed in Perkusich et al. (2013b), in which we collect data to quantify the relationships in the *Bayesian network* through a survey using a Likert-type scale (Likert, 1932). Afterwards, we statistically analyze the collected data and convert it to weighted expressions. Each relationship that has a common endpoint in the directed acyclic graph is represented by a weighted expression. Finally, the weighted expressions are translated into node probability tables using a truncated Normal distribution. We present the procedure used to define the node probability tables in Fig. 5. In Section 4, we show how we applied the proposed guidelines to model the software processes of a software development project that uses *Scrum*.

3.2. Stage 2: Bayesian Network Evaluation

This is the initial stage of the cyclic part of the procedure. After constructing the *Bayesian network* (i.e., finalizing Stage 1), the next step is to define how long the cycles will last (e.g., for iterative processes, a procedure's cycle could last one iteration) and when each stage will be executed. Afterwards, the model developer must evaluate if the given *Bayesian network's* structure (i.e., directed acyclic graph and probability functions) needs to be modified according to the project's current status and if necessary, modify it. If an existing *Bayesian network* is used to model the software processes of a given project, it is evaluated during this stage and if necessary, modified to suit the project's needs. In this case, if it is detected that too much effort is required to modify the existing *Bayesian network*, it could be better to go back to Stage 1 and build a new *Bayesian network* specifically for the given project. During the second and subsequent cycles, given that changes might occur in the software processes being used, the *Bayesian network* must be evaluated to check if it is up to date with the current project's status. Independent of the situation, this stage's output must be a *Bayesian network* that suits the current project's status.

3.3. Stage 3: Bayesian Network Data Input

At this stage, the user inputs data (i.e., evidences) to the *Bayesian network*. Ideally, all of the input nodes should have evidences. For the input nodes that data could not be measured from, and therefore, evidences could not be set, the uncertainty should be the same for all possible states. After inputting all evidences into

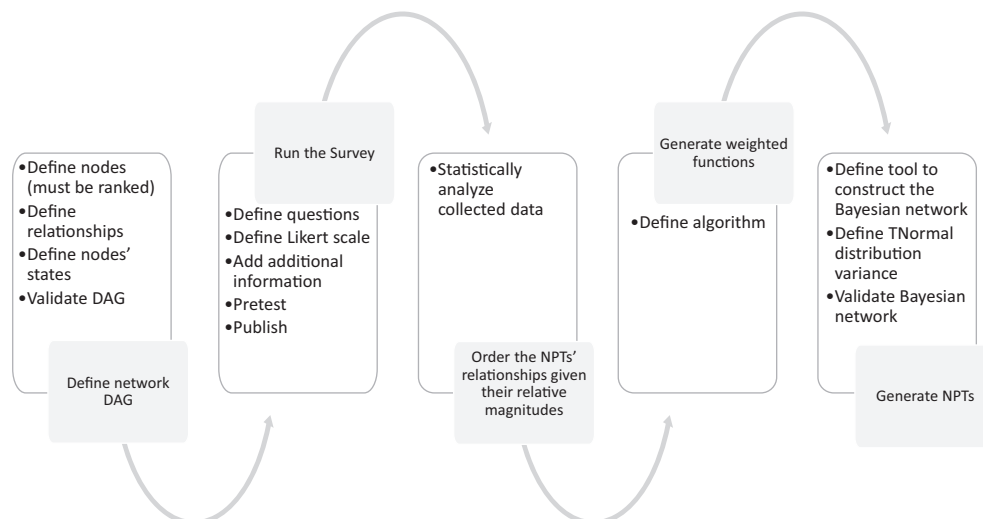


Fig. 5. Description of the steps required to define the node probability tables.

the model, the outputs should be calculated using a *Bayesian network's* specific tool such as GeNIe (<http://genie.sis.pitt.edu/>) or Netica (<http://www.norsys.com/>). The outputs are data with probability values for each node that represent the current status of key software process factors in the project. This stage's outputs are that the *Bayesian network* is fed with the project's current status data and outputs calculated. In this case, outputs relate to all nodes for which the user did not input an evidence.

3.4. Stage 4: Output analysis of the Bayesian network

At this stage, the project team should analyze the *Bayesian network's* calculated outputs to detect problems in the software process. This stage's goal is to assess the project's current situation and develop a plan to apply corrective and preventive actions to improve the project's chances of success. The project team might use sensitivity analysis to prioritize which problems should be resolved. This stage's output is a plan to execute the corrective and preventive actions identified during it.

3.5. Stage 5: Execution of corrective and preventive actions

During this stage, the plan defined in Stage 4 is executed and managed by the project manager or another project member. At the end of this stage, it is necessary to verify if the actions that were committed to were executed and what their consequences are.

4. An application of the proposed procedure to Scrum-based software development projects

To validate the procedure presented in Section 3, we applied it to build a *Bayesian network* that models a *Scrum*-based software development project and define its usage process to detect problems in the project's software processes. We validated the *Bayesian network* with simulated scenarios and the usage process with a case study that was executed in two projects of a software development company in Brazil.

The *Bayesian network* constructed includes *Scrum's* principles and rules, best practices used in the industry, and is limited to projects composed of only one *Scrum* team. Furthermore, we intended to build a *Bayesian network* for general *Scrum*-based software development projects from the perspective of the *ScrumMaster*. We did not try to represent all projects that use *Scrum*. In other words, the constructed *Bayesian network* should be used as a base to skip the first step of the procedure presented in Section 3 (i.e., *Bayesian Network Construction*) and reduce the effort needed to execute the second step (i.e., *Bayesian Network Evaluation*). The requirements for the *Bayesian network* and its usage process are: (i) be robust so to identify process problems in *Scrum*-based software development projects, (ii) the modification of *Bayesian network* (i.e., the second stage of the procedure) with acceptable cost, (iii) be useful to guide the team in its search for improvement, and (iv) have positive cost-benefit. To build the *Bayesian network*, we used AgenaRisk (www.agenarisk.com), which is a *Bayesian* inference engine that runs *Bayesian network* models and calculates the probabilities. The first version of a *Bayesian network* to detect process' problems in a *Scrum*-based software development project was published in Perkusich et al. (2013a).

4.1. Stage 1: Bayesian Network Construction

As described earlier, there are two challenges to building a *Bayesian network*: construct the directed acyclic graph and define the probability functions. To solve these challenges, we used the

guidelines described in Section 3. To define the directed acyclic graph, first we identified the elements of *F* and *R* (i.e., the nodes and edges). Afterwards we defined the elements of *N* (i.e., the node's states). For brevity, we will only show how we defined part of the directed acyclic graph. We created a webpage¹ to describe all the steps executed to build the *Bayesian network*.

To define the elements of *F* and *R*, the first step was to set *Process quality* as the output node. Afterwards, we researched the literature from respected practitioners (Cohn, 2009; Pichler, 2010; Sutherland & Schwaber, 2013) to decompose this node until we had nodes representing key software process factors that could be measured and easily observed. Instead of using the nodes suggested in the guideline described in Section 3 (i.e., *Software specification*, *Software design and implementation*, *Software validation* and *Software evolution*), we used key software process factors specific to *Scrum*: *Product increment quality*, *Product Owner overall quality* and *Work validation quality*.

In *Scrum*, work validation is performed during the *Sprint Retrospective meeting*. During this meeting, the *Product Owner* should check each sprint item's acceptance criteria and the sprint goal. The clients should give feedback on the presented results. Given this, we identified the child nodes of *Work validation quality*: *Acceptance criteria check*, *Sprint goal check*, and *Clients feedback*. To define the rest of the directed acyclic graph, we followed the same process. At the end of the process, we had the initial directed acyclic graph. *F* and *R* were composed of sixty-nine elements each.

Afterwards, the initial directed acyclic graph was presented to a group of eleven experts. They evaluated the directed acyclic graph and proposed some changes. For example, even though according to *scrumguide* the work validation should be executed during the *Sprint Review meeting*, in practice it is not the only situation where it occurs. Key clients are not available to participate in every *Scrum Review meeting*, but they give their feedback in separate meetings. We show the modifications performed on the directed acyclic graph to reflect these changes in Fig. 6. At the end of the process, we had the final directed acyclic graph. *F* and *R* were composed of seventy-three elements each. We show the final directed acyclic graph in Fig. 9.

The next step was to define the elements of *N* (i.e., the node's states). For each element of *F*, we defined three types of ordinal states: *Low*, *Medium* and *High*. *High* was defined as the desired state for the associated factor. Initially, for the associated probability p_i , we set all the independent nodes to 33%. The values of p_i will be defined after the team inputs evidences to the *Bayesian network* in step 3 of the procedure. This step completes the construction of the directed acyclic graph.

The next step was to define the probability functions. We followed the guidelines presented in Section 3 and used the approach proposed in Perkusich et al. (2013b). First, we published a survey to collect data from practitioners and order the relationships by their relative magnitudes. Afterwards, we developed an algorithm to generate weighted functions to be used in AgenaRisk to define the probability functions.

As described in Perkusich et al. (2013a), for each probability function that needed to be defined, we created a question on the survey. Fig. 7 shows an example of a survey question. To pre-test the survey, we published an initial draft of the survey using Surveytool (<http://www.surveytool.com>) to the same group of experts that helped in refining the list of key factors. They gave us feedback regarding the survey's wording, format, and content. As a result of the pre-test, we defined a template to be used to formulate all questions and decided to include a glossary explaining each tech-

¹ <http://mirkoperkusich.wordpress.com/2014/07/28/bayesian-network-construction/>

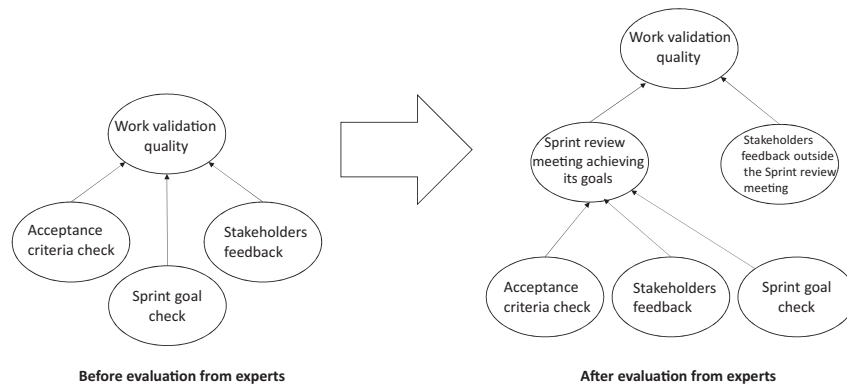


Fig. 6. Example of a modification after evaluation from experts.

What is the influence of the following factors on the factor "Sprint Review meeting achieves its goals"?

	Very High Influence	High Influence	Moderate Influence	Low Influence	Very Low Influence
Check if sprint goal was achieved	☐	☐	☐	☐	☐
Stakeholders feedback	☐	☐	☐	☐	☐
Check if acceptance criteria for sprint items were achieved	☐	☐	☐	☐	☐

Please, check the table below if you have any doubts.

Glossary	
Factor	Definition
Sprint Review meeting achieves its goals	Sprint Review meeting purpose is to give stakeholders the opportunity to check the product's current state and give feedback to ensure the project is following the correct path
Check if acceptance criteria for sprint items were achieved	During the Sprint Review meeting, the acceptance criteria for each product backlog item allocated for the sprint is inspected
Check if sprint goal was achieved	During the Sprint Review meeting, the PO checks if the sprint goal was achieved
Stakeholders feedback	During the Sprint Review meeting, the stakeholders check the product's current state and gives input regarding the increment created during the sprint and the product envisioned

Fig. 7. Example of a survey question.

nical term used on the survey. Furthermore, for simplicity, we decided to use a 5-point Likert scale to collect the practitioner's opinion for each relationship instead of an interval scale. Instead of giving us input to directly create the mathematical expressions to define the probability functions, the survey results gave us, for each question, data to be statistically analyzed to order the list of the relationships by its relative magnitude. After the survey was pre-tested, it was published using Surveytool which is still available at <http://www.surveytool.com/s/S666B24904>. The survey was published on Scrum-related groups on LinkedIn. Scrum.org was authorized to publish the survey on its Facebook Fan Page wall and ScrumAlliance published it on its Twitter account.

To order the relationships, we statistically analyzed the collected data using Friedman and Wilcoxon tests. The sample size (i.e., the number of respondents) was forty and composed of professionals with experience working on Scrum-based software projects in United States, Europe, India, and Brazil. Afterwards, we defined an algorithm to translate the ordered relationships into mathematical expressions (i.e., weighted expressions) to be used as input on AgenaRisk to generate the probability functions. Perkusich et al. (2013a) presents more details regarding this process.

4.2. Stage 2: Bayesian Network Evaluation

The first step was to determine the cycles' length. Since Scrum-based software development projects' schedules are organized in

sprints (i.e., iterations), the length of the procedures' cycles should be synchronized with the sprints in a way that the procedure's cycles start and finalize the same day as the sprints (i.e., $I = k * S$, where I is the cycle iteration's length, S is the sprints' length and $k \in \mathbb{N}$). This should be defined according to the project's necessity.

The first time this stage is executed, the first activity is to evaluate and if necessary, modify the Bayesian network constructed during Stage 1 to suit the particularities of the given project. This activity can be executed during a Scrum Retrospective meeting or a separate meeting. If executed during a separate meeting, the ScrumMaster and the Product Owner must participate. At least one developer should also participate. We recommend having people representing each of the Scrum's roles to have opinions from different perspectives.

4.3. Stage 3: Bayesian Network Data Input

As Stage 2, this stage is executed during the Scrum Retrospective meeting or a separate meeting. If executed during a separate meeting, the ScrumMaster and the Product Owner must participate. At least one developer should also participate. The ScrumMaster leads the observation of key software process factors that are represented in the Bayesian network. During the meeting, the team inputs data (i.e., evidences) to each node of the Bayesian network given the observation performed. The outputs should be calculated using AgenaRisk.

4.4. Stage 4: Output analysis of the Bayesian network

This stage is executed during the *Scrum Retrospective meeting*. Its goals are the same as of a regular *Scrum Retrospective meeting*. It will assess the project's current situation and develop a plan to apply corrective and preventive actions (i.e., action points plan) to improve the project's chances of success. During this meeting, the outputs calculated in Stage 3 can be used as a source of information to discuss the project's current situation. The *Bayesian network's* friendly user interface assists in identifying problems of the process. For each node (i.e., process key factor), it is possible to visualize its current situation and decide if it needs improvement. When the process' key factors that need improvement are identified, the team discusses action points for them. To assist in defining the action points plan, sensitivity analysis can be applied on the *Bayesian network* to identify the impact of the nodes on the output node, and assist in prioritizing the nodes (i.e., process key factors) that need improvement.

4.5. Stage 5: Execution of corrective and preventive actions

This stage is executed during the *sprints*. The *ScrumMaster* leads the team in applying the corrective and preventive actions. At the end of this stage, during a *Scrum Retrospective meeting* the *ScrumMaster* verifies if the planned action points were properly executed, their consequences, and updates the action points plan.

4.6. Validation

As stated earlier, applying the procedure defined a *Bayesian network* and its usage process for *Scrum*-based software development projects. We validated them separately. The *Bayesian network* validation followed two steps: node probability tables validation and *Bayesian network* outputs validation. The usage process validation consisted of a case study for two projects in a company located in Brazil. This paper limits itself to briefly describe the process used to test the node probability tables and present one simulated scenario used to test the *Bayesian network* outputs and state that the *Bayesian network* validation was successful. [Perkusich et al. \(2013a\)](#) presents more information regarding the *Bayesian network* validation. Section 4.6.2 presents details regarding the usage process validation.

4.6.1. Bayesian network validation

To test the node probability tables, we defined a set of inputs and the respective expected outputs for each table. Afterwards, we used AgenaRisk to calculate the output for each set of inputs. With the outputs calculated, we compared the actual outputs with the expected outputs. After analyzing the results, we concluded that they were acceptable.

We then defined ten simulated scenarios to validate the *Bayesian network* outputs. One of the scenarios represents a *Scrum*-based software development project that has a highly capable and organized development team, but does not have a committed and skilled Product Owner. Furthermore, the stakeholders do not collaborate closely with the team. The expected result for this project is for it to have a small chance of succeeding. The major reasons for the failure are the Product Owner's low work quality and lack of proper work validation. After defining this scenario, we defined the values for each input node and used AgenaRisk to calculate the *Bayesian network's* outputs. We present the values for the nodes *Project progress* and its parent nodes which are the most important nodes for the initial analysis in [Table 1](#). By analyzing the results on [Table 1](#), we can conclude that the results are acceptable because they raise a flag that the progress of the project will not go well. The biggest reasons for this are the poor work vali-

Table 1

Scenario results for highest level nodes in the *Bayesian network*.

Node	States		
	Bad (%)	Moderate (%)	Good (%)
Project progress	24	74	2
Product Owner overall work quality	71	29	0
Product increment quality	0	51	49
Work validation quality	0	50	50

Table 2

Problems detected during the first procedure iteration for the web application team and their possible impact on the process quality.

	Process quality		
	Bad (%)	Moderate (%)	Good (%)
Initial situation	0	13.532	86.466
Sprint Progress Monitoring = Good	0	9.727	90.273
Non-functional requirements = Good	0	11.908	88.091
Team velocity defined = Good	0	11.325	88.674
Visionary and doer PO = Good	0	12.980	87.019
3 to 5 top features defined = Good	0	12.639	87.360
Vision with critical attributes = Good	0	13.351	86.648
Clear acceptance criteria = Good	0	11.662	88.337
Independent PBI = Good	0	11.908	88.091
Members expertise = Good	0	13.381	86.618
Collective ownership = Good	0	13.381	86.618
Continuous improvement = Good	0	10.901	89.098
Pair programming = Good	0	12.977	87.022
Static code analysis = Good	0	13.298	86.701
Test coverage analysis = Good	0	13.162	86.837
Documentation = Good	0	13.380	86.619

Table 3

Problems detected during the first procedure iteration for the mobile team and their possible impact on the process quality.

	Process quality		
	Bad (%)	Moderate (%)	Good (%)
Initial situation	0	12.715	87.284
Sprint Progress Monitoring = Good	0	8.718	91.281
Team velocity defined = Good	0	10.212	89.787
Continuous improvement = Good	0	10.108	89.891
Visionary and doer PO = Good	0	12.177	87.822
3 to 5 top features defined = Good	0	11.804	88.195
Vision with critical attributes = Good	0	12.530	87.469
Clear acceptance criteria = Good	0	11.599	88.400
Independent PBI = Good	0	11.835	88.164
Non-functional requirements = Good	0	10.866	89.134
Collective ownership = Good	0	12.422	87.577
Team size = Good	0	12.422	87.577
Team physical distribution = Good	0	12.422	87.577
Pair programming = Good	0	12.011	87.988
Static code analysis = Good	0	12.417	87.582
Test coverage analysis = Good	0	12.131	87.868
Automated tests = Good	0	11.763	88.237
Documentation = Good	0	12.590	87.409

dation process and poor quality of work performed by the Product Owner. Given this, the project team could analyze the values of the nodes *Work validation*, *Product Owner work quality*, and their parents to improve the project's chances of success.

4.6.2. Bayesian network usage process validation

The next step was to validate the *Bayesian network* usage process. We performed a case study in two projects of a software company in Brazil. The company and projects were selected through

Table 4

Comparison between the calculated process quality before and after applying the procedure in the case study's projects.

	Server application Process quality			Mobile Process quality		
	Bad (%)	Moderate (%)	Good (%)	Bad (%)	Moderate (%)	Good (%)
Before	0	13.532	86.466	0	12.715	87.284
After	0	8.433	91.566	0	5.354	94.646

availability and willingness to run the case studies. The company had more than three years of experience using *Scrum* on most of its projects. The case study's duration was forty-five days (i.e., three *sprints*). The duration was defined given the minimum number of cycles we thought were enough to answer the study's research questions and the company's availability. Each project was composed of one development team. One project's scope was to develop a web application. This project's team was composed of six experienced developers. The other project's scope was to develop mobile clients for the web application. This

1. Is the procedure robust to identify process problems in Scrum-based software development projects?			
Yes	Needs some improvement	Needs a lot of improvement	No
2	0	0	0
Comments			
It was able to identify problems that have already raised our attention before using the procedure and some that we never noticed.			
We were able to identify some problems that were never noticed and that could have considerable impact on the project's success.			
2. Is the cost to modify \textit{Bayesian network} (i.e. second stage of the procedure) acceptable?			
Yes	Needs some improvement	Needs a lot of improvement	No
2	0	0	0
Comments			
We did not need to modify the model a lot.			
In our case, the model did not need many modifications.			
3. Is it useful to guide the team in its search for improvement?			
Yes	Needs some improvement	Needs a lot of improvement	No
2	0	0	0
Comments			
The model's analysis and data input helps the ScrumMaster to have a project's overview and to pay attention to aspects that might not be noticeable otherwise, specially to unexperienced ScrumMasters. Furthermore, the utilization of the model during Retrospective meetings helps in the identification of improvement opportunities.			
Its an additional weapon that I believe that can help in the identificaiton of improvement opportunities and problems. I believe that it is useful for ScrumMasters.			
4. Is the cost-benefit positive?			
Yes	Needs some improvement	Needs a lot of improvement	No
2	0	0	0
Comments			
The additional costs to use the procedure were just the time necessary to understand, modify and edit the model. Given that time that I worked with the procedure, I noticed that this team is easily compensated with positive impact that the procedure usage had in identifying improvement opportunities.			
The additional effort is some minutes during the sprints to analyze and input data to the model. In our case, this effort was compensated by the action points that were identified during the Retrospective meetings because of the procedure's usage.			

Fig. 8. Case study's survey responses.

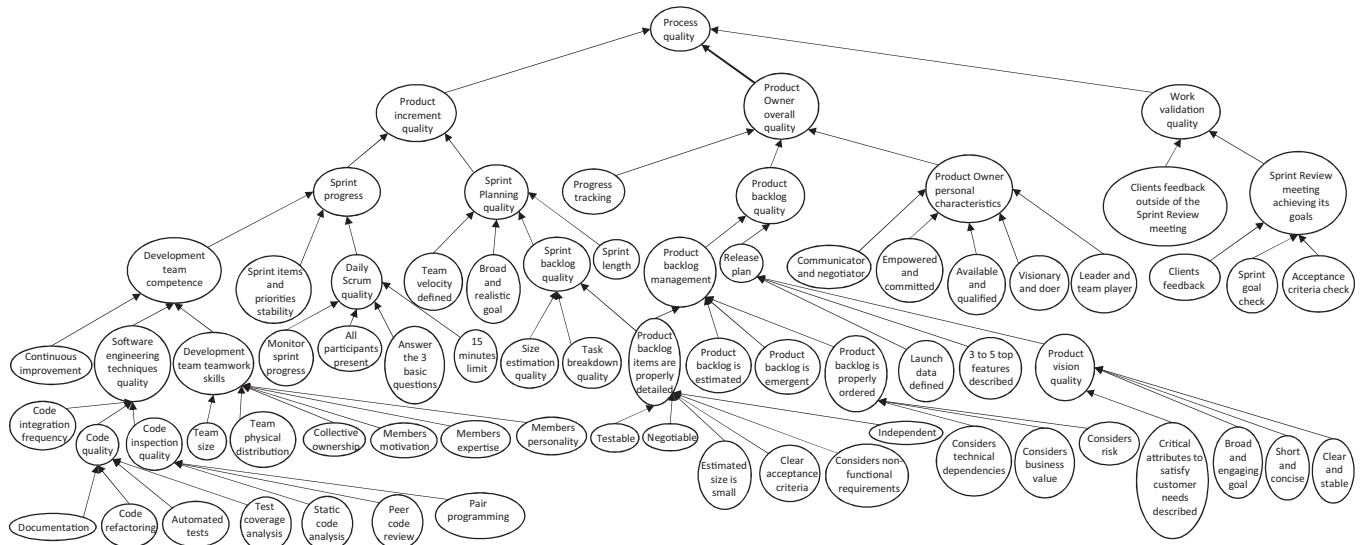


Fig. 9. Complete directed acyclic graph.

project's team was composed of five experienced developers. The case study's research questions were related to its requirements:

1. Is the procedure robust enough to identify process problems in Scrum-based software development projects?
2. Is the cost to modify the *Bayesian network* (i.e., second stage of the procedure) acceptable?
3. Is it useful to guide the team in its search for improvement?
4. Is the cost-benefit positive?

The study's propositions confirm each of the research questions. In both cases, we first met with the projects' *ScrumMaster*, the *Product Owner*, and one member of the development team to explain the procedure. Afterwards, without our presence, they met to execute Stages 1, 2 and 3. Both teams decided to skip Stages 1 and 2 and use the generic *Bayesian network* for Scrum-based software development projects that we proposed. At the end of this meeting, both teams had a *Bayesian network* representing their project's current status. Each meeting was time-boxed to one hour.

Afterwards, each team executed Stage 4 during a *Sprint Retrospective meeting*, which did not have its regular length modified. During the meetings, the *Bayesian networks'* outputs served as an extra source of information to detect problems. We present the most important problems identified with the *Bayesian network* and their possible impact on the process quality for the web application team in Table 2 and for the mobile team in Table 3. In both tables, the first line (i.e., *Initial situation*) presents the probability of each state of the node *Process quality* at the beginning of the procedure application. The forthcoming lines represent the probability of each state of the node *Process quality* if the detected problems are solved. For example, for the web application team, if the problem in *Sprint Progress Monitoring* is solved, the probability of the node *Process quality* be *Good* is 90.273%. For both projects, the problems with team velocity definition were due to recent changes in human resources. The teams and their respective *Product Owners* concluded that these changes would not happen again anytime soon so this item was ignored. As an action point to be executed during the next *sprint*, both teams decided to monitor the sprint's progress with *burndown charts*. The web application team committed to focus on continuous improvement by executing the action points committed on the *sprints*. For the mobile team, its *Product Owner* was advised to consider non-functional requirements to describe the *product backlog* items.

During the second *sprint*, both *ScrumMasters* led their respective teams to execute the action points committed during the last *Sprint Retrospective meeting*. This is part of the *ScrumMasters* job independent of using the procedure. At the end of the *sprint*, before the *Scrum* meetings, each project's *ScrumMaster*, *Product Owner*, and one member of the development team met to execute the Stage 3 of the procedure with the updated information. Each meeting was time-boxed to fifteen minutes.

During the second *sprint's Sprint Retrospective meeting*, the teams used the updated *Bayesian network* and the results of the *sprint's* action points execution to discuss the project's current status and define action points for the next *sprint*. By visualizing on the *Bayesian network* that they should improve their code quality, the mobile team decided to, during the next *sprint*, improve test coverage. The server team detected that the product had defects that could be avoided by improving the quality of the code inspection. They decided to reserve some time to fix legacy defects with the goal of not having relevant defects at the end of the *sprint*. Furthermore, the server team reserved some time to research for tools to apply static code analysis as part of the code review process.

During the third *sprint*, the same process was repeated. During the *Sprint Retrospective meeting*, the mobile team decided to continue working on improving test coverage. The server team, after fixing legacy bugs and finding static code analysis tools for the technologies they used, decided to as part of the peer review process, run static code analysis to decrease the probability of producing code with defects. We present the calculated improvement on process quality in Table 4. In both cases, the calculated process quality improved over 5%. Given the corrective actions that the teams executed, we conclude that calculated improvements – 5.100% for the web application team and 7.362% for the mobile team – reflect the reality. Even though the calculated improvements are not significantly high, given that we applied the procedure to mature teams, we conclude that the results are enough to demonstrate the procedure effectiveness.

Afterwards, we presented a survey containing the case study's research questions to the projects' *ScrumMasters*. To collect their opinions, the survey provided a 4-point Likert scale and then a comment box for each question. We show the collected data in Fig. 8. By analyzing the collected data, we concluded that all the case study's propositions were confirmed.

5. Conclusion and discussion

In this paper, we presented a procedure based in *Bayesian networks* to assist in detecting the process problems in software development projects. By increasing the efficiency of software processes, we increase the project's chances of success. The procedure consists of five stages: (i) *Bayesian Network Construction*, (ii) *Bayesian Network Evaluation*, (iii) *Bayesian Network Data Input*, (iv) *Bayesian network outputs' analysis* and (v) execution of corrective and preventive actions. Its goal is to expose the problems in software processes to help the team to improve the project's chances of success. For the first stage, we present a guideline to build *Bayesian networks* to model software development processes.

We applied the procedure on *Scrum*-based software development projects. The goal was to develop a generic model for these projects considering *Scrum*'s principles and rules as well as the industry's best practices. To execute the procedure's first stage (i.e., *Bayesian Network Construction*) we identified *Scrum*'s key software process factors and quantified their relationships. To identify the key software process factors, we researched the literature from respected practitioners and presented the identified factors to a group of experts. To quantify the relationships between the factors, we collected data from practitioners through an online survey. We then statistically analyzed the data, defined an algorithm to build weighted expressions for each relationship, and used the expressions to define the probability functions. Furthermore, we presented a guideline to apply stages (ii) through (iv) on *Scrum*-based software development projects. The *Bayesian network* was successfully validated through simulated scenarios. The procedure was successfully validated in two *Scrum*-based software development projects in a company located in Brazil.

The limitations of this study are related to *Bayesian Network Construction* and validations threats to validity. Regarding the empirical study (i.e., data collection through an online survey), there is no control regarding the people that answered the survey, their dedication, and the number of times each person responded. We attempted to minimize this threat by only publishing it to professional groups and having *Scrum Alliance* publish it to its members. The number of respondents might also not be enough to generate adequate data for a reliable model. Regarding the validation for *Scrum*-based software development projects, the number of simulated scenarios and case studies might not be enough to conclude that the procedure fits for any *Scrum*-based software development project.

Currently, we are performing case studies in more companies using the *Bayesian network* we developed for *Scrum*-based software development projects. Furthermore, one of the limitations of our procedure is to only rely on human inputs. So, we are researching a mechanism to use metrics as input to the *Bayesian network*.

For future works, we encourage researchers to apply the procedure to other methodologies and frameworks such as *RUP*, *Cleanroom* and *Kanban*, combine the procedure with Application Lifecycle Management processes and use the *Bayesian network* to model possible preventive and corrective actions (e.g., modify the project's *Product Owner*). Furthermore, other techniques such as Case Based Reasoning can be applied to calibrate the model given data collected throughout the project.

References

Abouelela, M., & Benedicenti, L. (2010). Bayesian network based xp process modelling. *International Journal of Software Engineering and Applications*, 1, 1–15.

Ahmed, M. A., & Muzaffar, Z. (2009). Handling imprecision and uncertainty in software development effort prediction: A type-2 fuzzy logic based framework. *Information and Software Technology*, 51, 640–654.

Anderson, D. J. (2010). *Successful evolutionary change for your technology business* (1st ed.). Blue Hole Press.

Bai, X., Huang, L., Zhang, H., & Koolmanojwong, S. (2012). Hybrid modeling and simulation for trustworthy software process management: A stakeholder-oriented approach. *Journal of Software: Evolution and Process*, 24, 721–740.

Bangs, O., & Willemin, P.-H. (2000). Top-down construction and repetitive structures representation in bayesian networks. In *Proceedings of the 13th international Florida artificial intelligence research society conference* (pp. 282–286). AAAI Press.

Beck, K. (2000). *Extreme programming explained: Embrace change* (1st ed.). Addison-Wesley.

Ben-Gal, I. (2007). Bayesian networks. In F. Ruggeri, R. Kenett, & F. W. Faltin (Eds.), *Encyclopedia of statistics in quality and reliability*. John Wiley and Sons.

Boehm, B. (1988). A spiral model of software development and enhancement. *Computer*, 21, 61–72.

Boehm, B. (2006). A view of 20th and 21st century software engineering. In *Proceedings of the 28th international conference on software engineering ICSE '06* (pp. 12–29). New York, NY, USA: ACM.

Büyükoçkan, G., & Ruan, D. (2010). Choquet integral based aggregation approach to software development risk assessment. *Information Sciences*, 180, 441–451.

Coad, P., de Luca, J., & Lefebvre, E. (1999). Java modeling. In *Color with UML: Enterprise components and process*. Prentice Hall PTR.

Cockburn, A. (2001). *Agile software development* (1st ed.). Addison-Wesley Professional.

Cohn, M. (2009). *Succeeding with agile: Software development using scrum* (1st ed.). Addison-Wesley Professional.

Das, B. (2004). Generating conditional probabilities for bayesian networks: Easing the knowledge acquisition problem. CoRR, cs.AI/0411034.

de Melo, A. C., & Sanchez, A. J. (2008). Software maintenance project delays prediction using bayesian networks. *Expert Systems with Applications*, 34, 908–919.

de Oliveira, M. A., Possamai, O., Dalla Valentina, L. V., & Flesch, C. A. (2012). Applying bayesian networks to performance forecast of innovation projects: A case study of transformational leadership influence in organizations oriented by projects. *Expert Systems with Applications*, 39, 5061–5070.

Diez, F. J. (1993). Parameter adjustment in bayes networks. The generalized noisy or-gate. In *Proceedings of the 9th conference on uncertainty in artificial intelligence* (pp. 99–105). Morgan Kaufmann.

Emam, K. E., & Kori, A. G. (2008). A replicated survey of it software project failures. *IEEE Software*, 25, 84–90.

Engel, A., & Last, M. (2007). Modeling software testing costs and risks using fuzzy logic paradigm. *Journal of Systems and Software*, 80, 817–835.

Fan, C.-F., & Yu, Y.-C. (2004). Bbn-based software project risk management. *Journal of Systems and Software*, 73, 193–203.

Fenton, N., Krause, P., & Neil, M. (2002). Software measurement: Uncertainty and causal modeling. *Software, IEEE*, 19, 116–122.

Fenton, N. E., Neil, M., & Caballero, J. G. (2007). Using ranked nodes to model qualitative judgments in bayesian networks. *IEEE Transactions on Knowledge and Data Engineering*, 19, 1420–1432.

Friedman, N., Geiger, D., & Goldszmidt, M. (1997). Bayesian network classifiers. *Machine Learning*, 29, 131–163.

Griffiths, M. (2012). *PMI-ACP exam prep*. RMC (Rita Mulcahy Companies) Publications.

Hearty, P., Fenton, N., Marquez, D., & Neil, M. (2009). Predicting project velocity in xp using a learning dynamic bayesian network model. *IEEE Transactions on Software Engineering*, 35, 124–137.

Heckerman, D. (1999). *Learning in graphical models. Chapter A tutorial on learning with bayesian networks*. Cambridge, MA, USA: MIT Press (pp. 301–354).

Highsmith, J. A. (1999). *Adaptive software development: A collaborative approach to managing complex systems* (1st ed.). Dorset House.

Houston, D. X., Mackulak, G. T., & Collofello, J. S. (2001). Stochastic simulation of risk factor potential effects for software development risk management. *Journal of Systems and Software*, 59, 247–257 (Software Process Simulation Modeling).

Huang, K., & Henrion, M. (1996). Efficient search-based inference for noisy-or belief networks: Topepsilon. In *Proceedings of the twelfth international conference on uncertainty in artificial intelligence UAI'96* (pp. 325–331). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc, pp. 325–331.

Humphrey, W. S. (1989). *Managing the software process*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The unified software development process* (1st ed.). Addison-Wesley Professional.

Jeet, K., Bhatia, N., & Minhas, R. S. (2011a). A bayesian network based approach for software defects prediction. *SIGSOFT Software Engineering Notes*, 36, 1–5.

Jeet, K., Bhatia, N., & Minhas, R. S. (2011b). A model for estimating the impact of low productivity on the schedule of a software development project. *SIGSOFT Software Engineering Notes*, 36, 1–6.

Kahen, G., Lehman, M., Ramil, J., & Wernick, P. (2001). System dynamics modelling of software evolution processes for policy investigation: Approach and example. *Journal of Systems and Software*, 59, 271–281. Software Process Simulation Modeling.

Kellner, M., Feiler, P., Finkelstein, A., Katayama, T., Osterweil, L., Penedo, M., & Rombach, H. (1991). Ispw-6 software process example. In *Proceedings of first international conference on software process* (pp. 176–186).

Kellner, M. I., Madachy, R. J., & Raffo, D. M. (1999). Software process simulation modeling: Why? what? how? *Journal of Systems and Software*, 46, 91–105.

Koller, D., & Pfeffer, A. (1997). Object-oriented bayesian networks. In *Proceedings of the thirteenth conference on uncertainty in artificial intelligence UAI'97* (pp. 302–313). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.

- Kouskouras, K. G., & Georgiou, A. C. (2007). A discrete event simulation model in the case of managing a software project. *European Journal of Operational Research*, 181, 374–389.
- Kruchten, P. (2003). *The rational unified process: An introduction* (3rd ed.). Addison-Wesley Professional.
- Kuppuswami, S., Vivekanandan, K., Ramaswamy, P., & Rodrigues, P. (2003). The effects of individual xp practices on software development effort. *SIGSOFT Software Engineering Notes*, 28, 1–6.
- Ladas, C. (2009). *Scrumban – Essays on kanban systems for lean software development* (1st ed.). Modus Cooperandi Press.
- Laskey, K. B., & Mahoney, S. M. (1997). Network fragments: Representing knowledge for constructing probabilistic models. In *Proceedings of the thirteenth conference on uncertainty in artificial intelligence UAI'97* (pp. 334–341). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Lee, E., Park, Y., & Shin, J. G. (2009). Large engineering project risk management using a bayesian belief network. *Expert Systems with Applications*, 36, 5880–5887.
- Likert, R. (1932). A technique for the measurement of attitudes. *Archives of Psychology*, 22, 1–55.
- Li, J., Li, M., Wu, D., & Song, H. (2012). An integrated risk measurement and optimization model for trustworthy software process management. *Information Sciences*, 191, 47–60.
- Mahoney, S. M., & Laskey, K. B. (1996). Network engineering for complex belief networks. In *Proceedings of the twelfth international conference on uncertainty in artificial intelligence UAI'96* (pp. 389–396). San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Martin, J. (1991). *Rapid application development*. Indianapolis, IN, USA: Macmillan Publishing Co., Inc.
- McCracken, D. D., & Jackson, M. A. (1982). Life cycle concept considered harmful. *SIGSOFT Software Engineering Notes*, 7, 29–32.
- Melis, M. (2006). A software process simulation model of extreme programming (Ph.D. thesis). Università di Cagliari.
- Mills, H., Dyer, M., & Linger, R. (1987). Cleanroom software engineering. *IEEE Software*, 4, 19–25.
- Nagy, A., Njima, M., & Mkrtchyan, L. (2010). A bayesian based method for agile software development release planning and project health monitoring. In *2010 2nd international conference on intelligent networking and collaborative systems (INCOS)* (pp. 192–199).
- Neil, M., Fenton, N., & Nielson, L. (2000). Building large-scale bayesian networks. *The Knowledge Engineering Review*, 15, 257–284.
- Osterweil, L. (1987). Software processes are software too. In *Proceedings of the 9th international conference on software engineering ICSE '87* (pp. 2–13). Los Alamitos, CA, USA: IEEE Computer Society Press.
- Osterweil, L. (1998). Jil and little-jil process programming languages. In V. Gruhn (Ed.), *Software process technology. Lecture notes in computer science* (Vol. 14). Berlin Heidelberg: Springer. 152–152.
- Pearl, J., & Russell, S. (1995). Bayesian networks. *Handbook of brain theory and neural networks*.
- Pendharkar, P., Subramanian, G., & Rodger, J. (2005). A probabilistic model for predicting software development effort. *IEEE Transactions on Software Engineering*, 31, 615–624.
- Perkusich, M., de Almeida, H. O., & Perkusich, A. (2013a). A model to detect problems on scrum-based software development projects. In *Proceedings of the 28th annual ACM symposium on applied computing SAC '13* (pp. 1037–1042). New York, NY, USA: ACM.
- Perkusich, M., Perkusich, A., & Almeida, H. (2013). Using survey and weighted functions to generate node probability tables for Bayesian networks. In *Proceedings of BRICS-CCI 2013*.
- Pichler, R. (2010). *Agile product management with scrum: Creating products that customers love* (1st ed.). Addison-Wesley Professional.
- Royce, W. W. (1970). Managing the development of large software systems: concepts and techniques. In *Proc. IEEE WESTCON* (pp. 1–9), Los Angeles. Reprinted in proceedings of the ninth international conference on software engineering (pp. 328–338), March 1987.
- Settas, D., Bibi, S., Sfetos, P., Stamelos, I., & Gerogiannis, V. (2006). Using bayesian belief networks to model software project management antipatterns. In *Fourth international conference on software engineering research, management and applications* (pp. 117–124), 2006.
- Sommerville, I. (2010). *Software engineering* (9th ed.). Addison-Wesley.
- Spasic, B., & Onggo, B. (2012). Agent-based simulation of the software development process: A case study at avl. In *simulation conference (WSC), proceedings of the 2012 winter* (pp. 1–11).
- Stamelos, I. (2010). Software project management anti-patterns. *Journal of Systems and Software*, 83, 52–59.
- Stamelos, I., Angelis, L., Dimou, P., & Sakellaris, E. (2003). On the use of bayesian belief networks for the prediction of software productivity. *Information and Software Technology*, 45, 51–60.
- Stapleton, J., & Constable, P. (1997). *DSDM: Dynamic systems development method: The method in practice* (1st ed.). Addison-Wesley Professional.
- Sutherland, J., & Schwaber, K. (2013). The scrum guide. <<https://www.scrum.org/Portals/0/Documents/Scrum%20Guides/2013/Scrum-Guide.pdf>>. Accessed: 2013-11-26.
- Uusitalo, L. (2007). Advantages and challenges of bayesian networks in environmental modelling. *Ecological Modelling*, 203, 312–318.
- Uzzafer, M. (2013). A simulation model for strategic management process of software projects. *Journal of Systems and Software*, 86, 21–37.
- VersionOne (2013). 7th annual state of agile development survey results. <<http://www.versionone.com/pdf/7th-Annual-State-of-Agile-Development-Survey.pdf>>. Accessed: 2013-11-26.
- Woodcock, J., Larsen, P. G., Bicarregui, J., & Fitzgerald, J. (2009). Formal methods: Practice and experience. *ACM Computing Surveys*, 41, 19:1–19:36.
- Wu, M., & Yan, H. (2009). Simulation in software engineering with system dynamics: A case study. *Journal of Software*, 4.
- Yuen, K. K. F., & Lau, H. C. (2011). A fuzzy group analytical hierarchy process approach for software quality assurance management: Fuzzy logarithmic least squares method. *Expert Systems with Applications*, 38, 10292–10302.
- Zhang, H., Kitchenham, B., & Pfahl, D. (2008). Reflections on 10 years of software process simulation modeling: A systematic review. In *Proceedings of the software process. 2008 International conference on making globally distributed software development a success story ICSP'08* (pp. 345–356). Berlin, Heidelberg: Springer-verlag.
- Zhang, L., Wu, X., Ding, L., Skibniewski, M. J., & Yan, Y. (2013). Decision support analysis for safety control in complex project environments based on bayesian networks. *Expert Systems with Applications*, 40, 4273–4282.
- Ziv, H., Richardson, D., & Klosch, R. (1996). The uncertainty principle in Software Engineering. Technical Report 96-33 University of California Irvine, California, USA.